

CS1200 / CSCI E-120 Lecture 6

Randomized algorithms

Fall 2026

Recommended reading: Hesterberg–Vadhan, Section 10.1-10.4

Announcements

- Mid-term next Wed (Lec 1-5, Pset 0-2, Section 0-2).
- Pset 2 due tomorrow.
- Pset 3 released.

Announcements

- Mid-term next Wed (Lec 1-5, Pset 0-2, Section 0-2).
- Pset 2 due tomorrow.
- Pset 3 released.
- Clarification on in-class reflection: we only drop grade if we find the combination (*did well on concept X in pset, did bad on concept X in in-class reflection*).
 - For example, if you did bad on pset 1 in class reflection, and also did not correctly discuss stability in pset 1, your grade will not be affected.

Course so far

- Lec 1-3: definition of computational problems, informal definition of algorithms, runtime of algorithms. Sorting problem has been our running example.

Course so far

- Lec 1-3: definition of computational problems, informal definition of algorithms, runtime of algorithms. Sorting problem has been our running example.
- Lec 4: Reduction between computational problems, notion of oracle.

Course so far

- Lec 1-3: definition of computational problems, informal definition of algorithms, runtime of algorithms. Sorting problem has been our running example.
- Lec 4: Reduction between computational problems, notion of oracle.
- Lec 5: Abstract data types, static and dynamic data structures, sorted arrays, binary search trees.

Course so far

- Lec 1-3: definition of computational problems, informal definition of algorithms, runtime of algorithms. Sorting problem has been our running example.
- Lec 4: Reduction between computational problems, notion of oracle.
- Lec 5: Abstract data types, static and dynamic data structures, sorted arrays, binary search trees.
- Today: randomization.

Median of key–value pairs

Let $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with $n \geq 1$. A median is a key-value pair (K, V) such that at most $\lfloor n/2 \rfloor$ keys are $< K$ and at most $\lfloor n/2 \rfloor$ keys strictly are $> K$.

Median of key–value pairs

Let $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with $n \geq 1$. A median is a key-value pair (K, V) such that at most $\lfloor n/2 \rfloor$ keys are $< K$ and at most $\lfloor n/2 \rfloor$ keys strictly are $> K$.

- If n is odd, choose the middle key in sorted order.
- If n is even, either of the two middle keys is a median.
- Duplicate keys can give several median key-value pairs.

Median of key–value pairs

Let $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with $n \geq 1$. A median is a key-value pair (K, V) such that at most $\lfloor n/2 \rfloor$ keys are $< K$ and at most $\lfloor n/2 \rfloor$ keys strictly are $> K$.

- If n is odd, choose the middle key in sorted order.
- If n is even, either of the two middle keys is a median.
- Duplicate keys can give several median key-value pairs.

Q: Median of $((1, a), (3, b), (2, c), (4, d))$?

Median of key–value pairs

Let $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, with $n \geq 1$. A median is a key-value pair (K, V) such that at most $\lfloor n/2 \rfloor$ keys are $< K$ and at most $\lfloor n/2 \rfloor$ keys strictly are $> K$.

- If n is odd, choose the middle key in sorted order.
- If n is even, either of the two middle keys is a median.
- Duplicate keys can give several median key-value pairs.

Q: Median of $((1, a), (3, b), (2, c), (4, d))$?

$(3, b)$ **and** $(2, c)$.

Why median?

Consider the keys

2, 3, 5, 4, 5, 3, 6.

Q: What are the mean and median?

Why median?

Consider the keys

2, 3, 5, 4, 5, 3, 6.

Q: What are the mean and median?

mean = 4, median = 4.

Why median?

Consider the keys

2, 3, 5, 4, 5, 3, 6.

Q: What are the mean and median?

mean = 4, median = 4.

Q: What happens if the last key changes from 6 to 600?

Why median?

Consider the keys

2, 3, 5, 4, 5, 3, 6.

Q: What are the mean and median?

mean = 4, median = 4.

Q: What happens if the last key changes from 6 to 600?

mean = $\frac{622}{7} \approx 88.9$, median = 4.

Medians are less sensitive to extreme values than means.

The selection problem

Input: a nonempty array A of n key–value pairs with keys in $\mathbb{N}$, and a rank $i \in [n] = \{0, \dots, n - 1\}$.

Output: a key-value pair whose key is an $(i + 1)$ st-smallest key.

The selection problem

Input: a nonempty array A of n key–value pairs with keys in $\mathbb{N}$, and a rank $i \in [n] = \{0, \dots, n-1\}$.

Output: a key-value pair whose key is an $(i+1)$ st-smallest key. This means, output (K_j, V_j) such that

$$|\{k : K_k < K_j\}| \leq i, \quad |\{k : K_k > K_j\}| \leq n - i - 1.$$

The selection problem

Input: a nonempty array A of n key–value pairs with keys in $\mathbb{N}$, and a rank $i \in [n] = \{0, \dots, n-1\}$.

Output: a key-value pair whose key is an $(i+1)$ st-smallest key. This means, output (K_j, V_j) such that

$$|\{k : K_k < K_j\}| \leq i, \quad |\{k : K_k > K_j\}| \leq n - i - 1.$$

Q: Which rank means a minimum? A maximum? A median?

The selection problem

Input: a nonempty array A of n key–value pairs with keys in $\mathbb{N}$, and a rank $i \in [n] = \{0, \dots, n-1\}$.

Output: a key-value pair whose key is an $(i+1)$ st-smallest key. This means, output (K_j, V_j) such that

$$|\{k : K_k < K_j\}| \leq i, \quad |\{k : K_k > K_j\}| \leq n - i - 1.$$

Q: Which rank means a minimum? A maximum? A median?

$$0, \quad n-1, \quad \left\lfloor \frac{n-1}{2} \right\rfloor.$$

Selection by sorting

Q: How can we solve SELECTION using a sorting algorithm?

Selection by sorting

Q: How can we solve SELECTION using a sorting algorithm?

1. Sort A by key in $O(n \log n)$ time.
2. Return the key-value pair at index i in $O(1)$ time.

Selection by sorting

Q: How can we solve SELECTION using a sorting algorithm?

1. Sort A by key in $O(n \log n)$ time.
2. Return the key-value pair at index i in $O(1)$ time.

Q: Is there a faster method?

Selection by sorting

Q: How can we solve SELECTION using a sorting algorithm?

1. Sort A by key in $O(n \log n)$ time.
2. Return the key-value pair at index i in $O(1)$ time.

Q: Is there a faster method?

Theorem

There is a **randomized algorithm** called *QuickSelect* that always solves SELECTION correctly and has worst-case *expected* running time $O(n)$.

Random choices

ps1.tex

ps1_experiments.py

ps2

```
25 def gen_random_arr(n, U):  
26     """generates random array given n and universe U"""  
27     arr = [(random.randint(0, U - 1), "foo") for _ in range(n)]  
28     return (arr, U, n)
```

Random choices

ps1.tex

ps1_experiments.py

ps2

```
25 def gen_random_arr(n, U):  
26     """generates random array given n and universe U"""  
27     arr = [(random.randint(0, U - 1), "foo") for _ in range(n)]  
28     return (arr, U, n)
```

Python's `random.randint(0, U-1)` draws an integer from $[U] = \{0, \dots, U - 1\}$.

Randomness in pseudocode

Add the command in the pseudocodes we have seen so far:

$$u = \text{random}(U), \quad U \geq 1.$$

Q: What does this command do?

Randomness in pseudocode

Add the command in the pseudocodes we have seen so far:

$$u = \text{random}(U), \quad U \geq 1.$$

Q: What does this command do?

- It assigns u a uniformly random element of $[U]$.
- Every value has probability $1/U$.
- Different calls produce independent random values.

Randomness in pseudocode

Add the command in the pseudocodes we have seen so far:

$$u = \text{random}(U), \quad U \geq 1.$$

Q: What does this command do?

- It assigns u a uniformly random element of $[U]$.
- Every value has probability $1/U$.
- Different calls produce independent random values.

We count this command as one basic operation.

* We have not introduced precise model of computation yet; we'll do so after midterm 1.

QuickSelect: the idea

Choose a uniformly random key-value pair, and call its key as *pivot*.

Q: Divide the array into sub-arrays:

QuickSelect: the idea

Choose a uniformly random key-value pair, and call its key as *pivot*.

Q: Divide the array into sub-arrays:

$A_{<} : \text{keys} < \text{pivot}$ $A_{=} : \text{keys} = \text{pivot}$ $A_{>} : \text{keys} > \text{pivot}$

QuickSelect: the idea

Choose a uniformly random key-value pair, and call its key as *pivot*.

Q: Divide the array into sub-arrays:

$A_{<} : \text{keys} < \text{pivot}$ $A_{=} : \text{keys} = \text{pivot}$ $A_{>} : \text{keys} > \text{pivot}$

- All keys in $A_{<}$ are less than all keys in $A_{=}$.
- All keys in $A_{=}$ are less than all keys in $A_{>}$.

QuickSelect: the idea

Choose a uniformly random key-value pair, and call its key as *pivot*.

Q: Divide the array into sub-arrays:

$A_{<} : \text{keys} < \text{pivot}$ $A_{=} : \text{keys} = \text{pivot}$ $A_{>} : \text{keys} > \text{pivot}$

- All keys in $A_{<}$ are less than all keys in $A_{=}$.
- All keys in $A_{=}$ are less than all keys in $A_{>}$.

Calculating the sizes of these arrays tells you where to look for rank i key.

QuickSelect: choosing and partitioning

Algorithm 1: QuickSelect(A, i)

Input: $n = |A| \geq 1$, keys in $\mathbb{N}$, and $0 \leq i < n$.

Output: A rank i key-value pair rank.

- 1 **if** $n = 1$ **then return** $A[0]$
 - 2 Set $p = \text{random}(n)$ and $\text{pivot} = K_p$
 - 3 Set $A_{<} =$ _____
 - 4 Set $A_{=} =$ _____
 - 5 Set $A_{>} =$ _____
 - 6 Set $n_{<} = |A_{<}|$, $n_{=} = |A_{=}|$, and $n_{>} = |A_{>}|$
-

Q: Which key-value pairs belong in $A_{<}$?

QuickSelect: choosing and partitioning

Algorithm 1: QuickSelect(A, i)

Input: $n = |A| \geq 1$, keys in $\mathbb{N}$, and $0 \leq i < n$.

Output: A rank i key-value pair rank.

- 1 if $n = 1$ then return $A[0]$
 - 2 Set $p = \text{random}(n)$ and $\text{pivot} = K_p$
 - 3 Set $A_{<} =$ the key-value pairs of A with keys $<$ pivot
 - 4 Set $A_{=} =$ _____
 - 5 Set $A_{>} =$ _____
 - 6 Set $n_{<} = |A_{<}|$, $n_{=} = |A_{=}|$, and $n_{>} = |A_{>}|$
-

Q: How should we handle key-value pairs tied with the pivot?

QuickSelect: choosing and partitioning

Algorithm 1: QuickSelect(A, i)

Input: $n = |A| \geq 1$, keys in $\mathbb{N}$, and $0 \leq i < n$.

Output: A rank i key-value pair rank.

- 1 if $n = 1$ then return $A[0]$
 - 2 Set $p = \text{random}(n)$ and $\text{pivot} = K_p$
 - 3 Set $A_{<} =$ the key-value pairs of A with keys $<$ pivot
 - 4 Set $A_{=} =$ the key-value pairs of A with key = pivot
 - 5 Set $A_{>} =$ _____
 - 6 Set $n_{<} = |A_{<}|$, $n_{=} = |A_{=}|$, and $n_{>} = |A_{>}|$
-

Q: What remains for $A_{>}$?

QuickSelect: choosing and partitioning

Algorithm 1: QuickSelect(A, i)

Input: $n = |A| \geq 1$, keys in $\mathbb{N}$, and $0 \leq i < n$.

Output: A rank i key-value pair rank.

- 1 if $n = 1$ then return $A[0]$
 - 2 Set $p = \text{random}(n)$ and $\text{pivot} = K_p$
 - 3 Set $A_{<} =$ the key-value pairs of A with keys $<$ pivot
 - 4 Set $A_{=} =$ the key-value pairs of A with key $=$ pivot
 - 5 Set $A_{>} =$ the key-value pairs of A with key $>$ pivot
 - 6 Set $n_{<} = |A_{<}|$, $n_{=} = |A_{=}|$, and $n_{>} = |A_{>}|$
-

The three sub-arrays include all the key-value pairs, and $n_{<} + n_{=} + n_{>} = n$.

QuickSelect: choosing the recursive call

View of the sub-arrays in the sorted order:

$\underbrace{A}_{A_{<}}$ $\underbrace{A}_{A_{=}}$ $\underbrace{A}_{A_{>}}$.
 $n_{<}$ key-value pairs $n_{=}$ key-value pairs $n_{>}$ key-value pairs

Algorithm 2: QuickSelect (continued)(A, i)

1 if $i < n_{<}$ then _____
2 else if $i \geq n_{<} + n_{=}$ then _____
3 else _____

Q: If rank $i < n_{<}$, what do we do?

QuickSelect: choosing the recursive call

View of the sub-arrays in the sorted order:

$\underbrace{A_{<}}_{n_{<} \text{ key-value pairs}} \quad \underbrace{A_{=}}_{n_{=} \text{ key-value pairs}} \quad \underbrace{A_{>}}_{n_{>} \text{ key-value pairs}} \cdot$

Algorithm 2: QuickSelect (continued)(A, i)

1 if $i < n_{<}$ then **return** QuickSelect($A_{<}, i$)

2 else if $i \geq n_{<} + n_{=}$ then _____

3 else _____

Q: If rank $i \geq n_{<} + n_{=}$, what do we do?

QuickSelect: choosing the recursive call

View of the sub-arrays in the sorted order:

$\underbrace{A_{<}}_{n_{<} \text{ key-value pairs}} \quad \underbrace{A_{=}}_{n_{=} \text{ key-value pairs}} \quad \underbrace{A_{>}}_{n_{>} \text{ key-value pairs}} .$

Algorithm 2: QuickSelect (continued)(A, i)

1 if $i < n_{<}$ then **return** QuickSelect($A_{<}, i$)
2 else if $i \geq n_{<} + n_{=}$ then **return** QuickSelect($A_{>}, i - n_{<} - n_{=}$)
3 else _____

Q: Otherwise?

QuickSelect: choosing the recursive call

View of the sub-arrays in the sorted order:

$\underbrace{A_{<}}_{n_{<} \text{ key-value pairs}} \quad \underbrace{A_{=}}_{n_{=} \text{ key-value pairs}} \quad \underbrace{A_{>}}_{n_{>} \text{ key-value pairs}} .$

Algorithm 2: QuickSelect (continued)(A, i)

```
1 if  $i < n_{<}$  then return QuickSelect( $A_{<}, i$ )
2 else if  $i \geq n_{<} + n_{=}$  then return QuickSelect( $A_{>}, i - n_{<} - n_{=}$ )
3 else return  $A_{=}[0]$ 
```

Any key-value pair in $A_{=}$ is valid output.

Example: the first pivot

Keys (ignore values):

$$A = (20, 30, 50, 60, 30, 50, 40), \quad i = 3.$$

Q: We choose index $p = 2$, so $\text{pivot} = 50$. What are the sub-arrays?

Example: the first pivot

Keys (ignore values):

$$A = (20, 30, 50, 60, 30, 50, 40), \quad i = 3.$$

Q: We choose index $p = 2$, so pivot = 50. What are the sub-arrays?

$$A_{<} = (20, 30, 30, 40), \quad A_{=} = (50, 50), \quad A_{>} = (60).$$

Example: the first pivot

Keys (ignore values):

$$A = (20, 30, 50, 60, 30, 50, 40), \quad i = 3.$$

Q: We choose index $p = 2$, so pivot = 50. What are the sub-arrays?

$$A_{<} = (20, 30, 30, 40), \quad A_{=} = (50, 50), \quad A_{>} = (60).$$

Q: What call do we make next?

Example: the first pivot

Keys (ignore values):

$$A = (20, 30, 50, 60, 30, 50, 40), \quad i = 3.$$

Q: We choose index $p = 2$, so $\text{pivot} = 50$. What are the sub-arrays?

$$A_{<} = (20, 30, 30, 40), \quad A_{=} = (50, 50), \quad A_{>} = (60).$$

Q: What call do we make next?

$$i = 3 < n_{<} = 4 \implies \text{QuickSelect}((20, 30, 30, 40), 3).$$

We still want rank 3, the fourth-smallest key in this smaller array.

Example: the next pivot

Consider `QuickSelect((20, 30, 30, 40), 3)`. Suppose its random pivot key is now `pivot = 30`.]

Q: Look at the subarray:

Example: the next pivot

Consider `QuickSelect((20, 30, 30, 40), 3)`. Suppose its random pivot key is now `pivot = 30`.]

Q: Look at the subarray:

$$A_{<} = (20), \quad A_{=} = (30, 30), \quad A_{>} = (40).$$

Example: the next pivot

Consider $\text{QuickSelect}((20, 30, 30, 40), 3)$. Suppose its random pivot key is now $\text{pivot} = 30$.]

Q: Look at the subarray:

$$A_{<} = (20), \quad A_{=} = (30, 30), \quad A_{>} = (40).$$

$$i = 3 \geq n_{<} + n_{=} = 1 + 2, \quad i - n_{<} - n_{=} = 0.$$

Example: the next pivot

Consider $\text{QuickSelect}((20, 30, 30, 40), 3)$. Suppose its random pivot key is now $\text{pivot} = 30$.]

Q: Look at the subarray:

$$A_{<} = (20), \quad A_{=} = (30, 30), \quad A_{>} = (40).$$

$$i = 3 \geq n_{<} + n_{=} = 1 + 2, \quad i - n_{<} - n_{=} = 0.$$

$\text{QuickSelect}((40), 0)$ returns 40.

QuickSelect: correctness

We prove correctness for *every* sequence of pivot choices, using strong induction on the array size.

$P(k)$: For every array of size k and every valid rank i ,
QuickSelect(A, i) returns a valid key-value pair of rank i .

QuickSelect: correctness

We prove correctness for *every* sequence of pivot choices, using strong induction on the array size.

$P(k)$: For every array of size k and every valid rank i ,
QuickSelect(A, i) returns a valid key-value pair of rank i .

- Base case $k = 1$: the only valid rank is 0, and we return $A[0]$.

QuickSelect: correctness

We prove correctness for *every* sequence of pivot choices, using strong induction on the array size.

$P(k)$: For every array of size k and every valid rank i ,
QuickSelect(A, i) returns a valid key-value pair of rank i .

- Base case $k = 1$: the only valid rank is 0, and we return $A[0]$.
- Inductive step: assume $P(k)$ for all $k < n$.

QuickSelect: correctness

We prove correctness for *every* sequence of pivot choices, using strong induction on the array size.

$P(k)$: For every array of size k and every valid rank i ,
 QuickSelect(A, i) returns a valid key-value pair of rank i .

- Base case $k = 1$: the only valid rank is 0, and we return $A[0]$.
- Inductive step: assume $P(k)$ for all $k < n$.
- Note: each recursive call considers a smaller sub-array.

QuickSelect: correctness

Q: Why does each branch request the correct rank?

QuickSelect: correctness

Q: Why does each branch request the correct rank?

- $0 \leq i < n_{<}$: the desired key-value pair is inside $A_{<}$. By induction, the recursive call is correct.

QuickSelect: correctness

Q: Why does each branch request the correct rank?

- $0 \leq i < n_{<}$: the desired key-value pair is inside $A_{<}$. By induction, the recursive call is correct.
- $n_{<} \leq i < n_{<} + n_{=}$: the desired key-value pair is in $A_{=}$ and the output $A_{=}[0]$ is valid.

QuickSelect: correctness

Q: Why does each branch request the correct rank?

- $0 \leq i < n_{<}$: the desired key-value pair is inside $A_{<}$. By induction, the recursive call is correct.
- $n_{<} \leq i < n_{<} + n_{=}$: the desired key-value pair is in $A_{=}$ and the output $A_{=}[0]$ is valid.
- $n_{<} + n_{=} \leq i < n$: skip the first $n_{<} + n_{=}$ positions. The desired rank inside $A_{>}$ is $i - n_{<} - n_{=}$. By induction, the recursive call is correct.

QuickSelect: correctness

Q: Why does each branch request the correct rank?

- $0 \leq i < n_{<}$: the desired key-value pair is inside $A_{<}$. By induction, the recursive call is correct.
- $n_{<} \leq i < n_{<} + n_{=}$: the desired key-value pair is in $A_{=}$ and the output $A_{=}[0]$ is valid.
- $n_{<} + n_{=} \leq i < n$: skip the first $n_{<} + n_{=}$ positions. The desired rank inside $A_{>}$ is $i - n_{<} - n_{=}$. By induction, the recursive call is correct.

Q: What happens if every key equals the pivot?

QuickSelect: correctness

Q: Why does each branch request the correct rank?

- $0 \leq i < n_{<}$: the desired key-value pair is inside $A_{<}$. By induction, the recursive call is correct.
- $n_{<} \leq i < n_{<} + n_{=}$: the desired key-value pair is in $A_{=}$ and the output $A_{=}[0]$ is valid.
- $n_{<} + n_{=} \leq i < n$: skip the first $n_{<} + n_{=}$ positions. The desired rank inside $A_{>}$ is $i - n_{<} - n_{=}$. By induction, the recursive call is correct.

Q: What happens if every key equals the pivot?

We return from $A_{=}$ after one partitioning pass.

QuickSelect: various runs

Suppose the keys are distinct and we ask for the maximum, $i = n - 1$.

Q: What happens if every pivot is the smallest remaining key?

QuickSelect: various runs

Suppose the keys are distinct and we ask for the maximum, $i = n - 1$.

Q: What happens if every pivot is the smallest remaining key? The recursive array sizes are

$$n, n - 1, n - 2, \dots, 1.$$

QuickSelect: various runs

Suppose the keys are distinct and we ask for the maximum, $i = n - 1$.

Q: What happens if every pivot is the smallest remaining key? The recursive array sizes are

$$n, n - 1, n - 2, \dots, 1.$$

Each partitioning pass takes time proportional to the current array size.

$$n + (n - 1) + \dots + 1 = \Theta(n^2).$$

QuickSelect: various runs

Suppose the keys are distinct and we ask for the maximum, $i = n - 1$.

Q: What happens if every pivot is the smallest remaining key? The recursive array sizes are

$$n, n - 1, n - 2, \dots, 1.$$

Each partitioning pass takes time proportional to the current array size.

$$n + (n - 1) + \dots + 1 = \Theta(n^2).$$

Q: But what if the pivot is the largest key?

QuickSelect: various runs

Suppose the keys are distinct and we ask for the maximum, $i = n - 1$.

Q: What happens if every pivot is the smallest remaining key? The recursive array sizes are

$$n, n - 1, n - 2, \dots, 1.$$

Each partitioning pass takes time proportional to the current array size.

$$n + (n - 1) + \dots + 1 = \Theta(n^2).$$

Q: But what if the pivot is the largest key?

One iteration suffices with $O(n)$ runtime.

The running time depends on the random choices.

QuickSelect: expected shrinkage

Let M be the size of the next recursive array, or 0 if we return directly.

QuickSelect: expected shrinkage

Let M be the size of the next recursive array, or 0 if we return directly.

$$M \leq \max\{n_<, n_>\}.$$

Q: How large is this bound on average over a uniformly random pivot?

QuickSelect: expected shrinkage

Let M be the size of the next recursive array, or 0 if we return directly.

$$M \leq \max\{n_<, n_>\}.$$

Q: How large is this bound on average over a uniformly random pivot?

$$\mathbb{E}[M] \leq \frac{3n}{4}.$$

(We will not do this calculation; all you need to know is that expected size drops).

Intuitively, this means that the expected runtime satisfies $T_{\text{expected}}(n) \leq cn + T_{\text{expected}}(3n/4)$.

By unrolling, this can be solved to give $T_{\text{expected}}(n) \leq 4cn$.

Las Vegas algorithms

A *Las Vegas algorithm* always returns a correct answer. Its running time may depend on its random choices.

Las Vegas algorithms

A *Las Vegas algorithm* always returns a correct answer. Its running time may depend on its random choices.

- Unlucky random choices may cause a long run.
- We usually bound the *expected* running time.

Las Vegas algorithms

A *Las Vegas algorithm* always returns a correct answer. Its running time may depend on its random choices.

- Unlucky random choices may cause a long run.
- We usually bound the *expected* running time.

Q: Why is QuickSelect a Las Vegas algorithm?

Las Vegas algorithms

A *Las Vegas algorithm* always returns a correct answer. Its running time may depend on its random choices.

- Unlucky random choices may cause a long run.
- We usually bound the *expected* running time.

Q: Why is QuickSelect a Las Vegas algorithm?

Every pivot sequence gives a correct answer, and the expected running time is $O(n)$.

An individual run may still take $\Theta(n^2)$ time.

Worst-case expected running time

For a fixed input x , let $T_B(x)$ be the random variable giving algorithm B 's running time.

Worst-case expected running time

For a fixed input x , let $T_B(x)$ be the random variable giving algorithm B 's running time.

$$T_{\text{expected}}(n) = \max_{x: \text{size}(x) \leq n} \mathbb{E}[T_B(x)].$$

Worst-case expected running time

For a fixed input x , let $T_B(x)$ be the random variable giving algorithm B 's running time.

$$T_{\text{expected}}(n) = \max_{x: \text{size}(x) \leq n} \mathbb{E}[T_B(x)].$$

For a nonnegative integer-valued random variable Z ,

$$\mathbb{E}[Z] = \sum_{z \in \mathbb{N}} z \cdot \Pr[Z = z].$$

Fix an input, average over the random choices, then take the worst input.

Monte Carlo algorithms

A *Monte Carlo algorithm* B always runs within a chosen time bound, but may give an incorrect answer with small probability.

Monte Carlo algorithms

A *Monte Carlo algorithm* B always runs within a chosen time bound, but may give an incorrect answer with small probability. For a computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$, error probability at most p means that, for every input $x \in \mathcal{I}$,

Monte Carlo algorithms

A *Monte Carlo algorithm* B always runs within a chosen time bound, but may give an incorrect answer with small probability. For a computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$, error probability at most p means that, for every input $x \in \mathcal{I}$,

- If $f(x) \neq \emptyset$, then

$$\Pr[B(x) \in f(x)] \geq 1 - p.$$

Monte Carlo algorithms

A *Monte Carlo algorithm* B always runs within a chosen time bound, but may give an incorrect answer with small probability. For a computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$, error probability at most p means that, for every input $x \in \mathcal{I}$,

- If $f(x) \neq \emptyset$, then

$$\Pr[B(x) \in f(x)] \geq 1 - p.$$

- If $f(x) = \emptyset$, then

$$\Pr[B(x) = \perp] \geq 1 - p.$$

Las Vegas and Monte Carlo

Q: Which guarantee can depend on being lucky?

Las Vegas and Monte Carlo

Q: Which guarantee can depend on being lucky?

	Las Vegas	Monte Carlo
Answer	Always correct	May be incorrect
Running time	Bound on expectation	Bound on every run

Las Vegas and Monte Carlo

Q: Which guarantee can depend on being lucky?

	Las Vegas	Monte Carlo
Answer	Always correct	May be incorrect
Running time	Bound on expectation	Bound on every run

Both guarantees must hold for *every input*. The probability comes from the algorithm's random choices.

Las Vegas and Monte Carlo

Q: Which guarantee can depend on being lucky?

	Las Vegas	Monte Carlo
Answer	Always correct	May be incorrect
Running time	Bound on expectation	Bound on every run

Both guarantees must hold for *every input*. The probability comes from the algorithm's random choices.

QuickSelect is Las Vegas. Next: a Monte Carlo sampling algorithm.

EstimateProportion

A pollster wants to estimate the fraction of people who prefer singing to dancing. **Input:**

$A = (t_0, \dots, t_{n-1})$, where $n \geq 1$, $t_j \in \{S, D\}$, and an error tolerance $\varepsilon \in (0, 1)$.

EstimateProportion

A pollster wants to estimate the fraction of people who prefer singing to dancing. **Input:**

$A = (t_0, \dots, t_{n-1})$, where $n \geq 1$, $t_j \in \{S, D\}$, and an error tolerance $\varepsilon \in (0, 1)$. Let the true fraction be

$$\theta = \frac{|\{j : t_j = S\}|}{n}.$$

Output:

EstimateProportion

A pollster wants to estimate the fraction of people who prefer singing to dancing. **Input:**

$A = (t_0, \dots, t_{n-1})$, where $n \geq 1$, $t_j \in \{S, D\}$, and an error tolerance $\varepsilon \in (0, 1)$. Let the true fraction be

$$\theta = \frac{|\{j : t_j = S\}|}{n}.$$

Output:

A rational number $r \in [0, 1]$ such that

$$|r - \theta| \leq \varepsilon.$$

Estimating the proportion by sampling

Q: Can we estimate θ without reading the entire array?

Estimating the proportion by sampling

Q: Can we estimate θ without reading the entire array?

Algorithm 3: EstimateBySampling(A, ε)

```
1 Set  $m = \lceil 25/\varepsilon^2 \rceil$  and  $s = 0$ 
2 foreach  $t = 1, \dots, m$  do
3   | Draw  $j = \text{random}(n)$  independently
4   | if  $A[j] = S$  then Set  $s = s + 1$ 
5 return  $r = s/m$ 
```

Estimating the proportion by sampling

Q: Can we estimate θ without reading the entire array?

Algorithm 3: EstimateBySampling(A, ε)

```
1 Set  $m = \lceil 25/\varepsilon^2 \rceil$  and  $s = 0$ 
2 foreach  $t = 1, \dots, m$  do
3   Draw  $j = \text{random}(n)$  independently
4   if  $A[j] = S$  then Set  $s = s + 1$ 
5 return  $r = s/m$ 
```

Sampling is *with replacement*: the same index may be chosen again. Assume a random index and access to $A[j]$ each take $O(1)$ time.

The sampling guarantee

With $m = \lceil 25/\varepsilon^2 \rceil$ independent samples, a probability calculation gives

$$\Pr[|r - \theta| \leq \varepsilon] \geq 0.99.$$

The sampling guarantee

With $m = \lceil 25/\varepsilon^2 \rceil$ independent samples, a probability calculation gives

$$\Pr[|r - \theta| \leq \varepsilon] \geq 0.99.$$

- This holds for every fixed array A .
- The algorithm always takes $O(1/\varepsilon^2)$ time in our access model.
- It is a Monte Carlo algorithm: some random samples give a bad estimate.

Power of randomness

Are there problems with faster randomized solutions than the fastest deterministic ones?

Power of randomness

Are there problems with faster randomized solutions than the fastest deterministic ones?

- **SELECTION:** We discussed the $O(n)$ time Las-Vegas algorithm. There is a deterministic algorithm with runtime $O(n)$, which uses a more complicated strategy to choose a pivot.

Power of randomness

Are there problems with faster randomized solutions than the fastest deterministic ones?

- **SELECTION:** We discussed the $O(n)$ time Las-Vegas algorithm. There is a deterministic algorithm with runtime $O(n)$, which uses a more complicated strategy to choose a pivot.
- **Primality testing:** Given an integer of size n , check if it is prime. There is an $O(n^3)$ -time Monte Carlo algorithm, $O(n^4)$ -time Las Vegas algorithm, and a $O(n^6)$ -time deterministic algorithm (proven in the paper “Primes is in P”).

Power of randomness

Are there problems with faster randomized solutions than the fastest deterministic ones?

- **SELECTION:** We discussed the $O(n)$ time Las-Vegas algorithm. There is a deterministic algorithm with runtime $O(n)$, which uses a more complicated strategy to choose a pivot.
- **Primality testing:** Given an integer of size n , check if it is prime. There is an $O(n^3)$ -time Monte Carlo algorithm, $O(n^4)$ -time Las Vegas algorithm, and a $O(n^6)$ -time deterministic algorithm (proven in the paper “Primes is in P”).
- **Identity Testing:** Given some arithmetic expression (like $12875^{8091761235} \cdot (15676 + 91247^{259}) - 967012^{8107069871}$), check if it is equal to zero. This has an $O(n^2)$ -time Monte Carlo algorithm, and the best currently-known deterministic algorithm runs in $2^{O(n)}$.

Power of randomness

Are there problems with faster randomized solutions than the fastest deterministic ones?

- **SELECTION:** We discussed the $O(n)$ time Las-Vegas algorithm. There is a deterministic algorithm with runtime $O(n)$, which uses a more complicated strategy to choose a pivot.
- **Primality testing:** Given an integer of size n , check if it is prime. There is an $O(n^3)$ -time Monte Carlo algorithm, $O(n^4)$ -time Las Vegas algorithm, and a $O(n^6)$ -time deterministic algorithm (proven in the paper “Primes is in P”).
- **Identity Testing:** Given some arithmetic expression (like $12875^{8091761235} \cdot (15676 + 91247^{259}) - 967012^{8107069871}$), check if it is equal to zero. This has an $O(n^2)$ -time Monte Carlo algorithm, and the best currently-known deterministic algorithm runs in $2^{O(n)}$.

Conjecture:

randomness is not necessary for polynomial-time computation.